

Take-Home Challenge

Distributed Inventory Sync

The Objective

Build a "Live Distributed Inventory" app. Imagine two warehouse workers using the app on different phones. The app must keep their screens in sync across three scenarios:

- Both have internet — Cloud Sync
- One loses internet — Offline Queue
- Both lose internet but are on the same Wi-Fi — Local Peer-to-Peer Sync

Functional Requirements

The UI (Keep it simple)

- A screen displaying a list of inventory items (e.g., Item Name and Quantity).
- Each item should have "+" and "-" buttons to adjust the quantity.
- A visual indicator of the app's current sync state:  Online,  Local Network Only,  Offline.

Phase 1: The Real-Time Cloud Sync (MQTT)

- The app must communicate over MQTT using a free, public broker (e.g., test.mosquitto.org).
- When a user changes a quantity, the app publishes a message to a specific topic.
- The app must listen to that same topic and update the UI instantly when a payload arrives from the other device.

Phase 2: Offline-First & Queuing

- If the device loses internet connection, the UI must still allow the user to change quantities.
- These changes must be saved to a local database (e.g., SQLite, Hive, Isar).
- Write a Sync Manager that queues offline changes. When the MQTT connection is re-established, the app should automatically publish queued messages to the broker.

Phase 3: The Local Fallback (UDP)

- If the MQTT broker is unreachable but both phones are on the same local Wi-Fi router, the app should broadcast inventory updates over UDP (User Datagram Protocol).

- The app should listen for incoming UDP packets and update the local UI when an inventory update is received.

Constraints

No Firebase/BaaS: You must write the MQTT and UDP implementations yourself using standard networking packages.

State Management: Cleanly separate your networking/sync logic from your UI widgets.

Required: The 5-Minute Video Walkthrough

Please record a screen-share video (Loom, Zoom, etc.) answering the following:

1. Demonstration

Show us the app running. Demonstrate the MQTT or UDP sync with two instances side-by-side.

2. Architecture

Walk us through your folder structure. How do you decide when to use MQTT vs. UDP vs. the Offline Queue?

How to Submit

Create a private GitHub repository and invite the reviewer(s).

Update the README.md with any setup instructions.

Include the link to your video walkthrough at the very top of your README.

Good luck! We look forward to seeing your solution.